Devkit v2: a box of your own

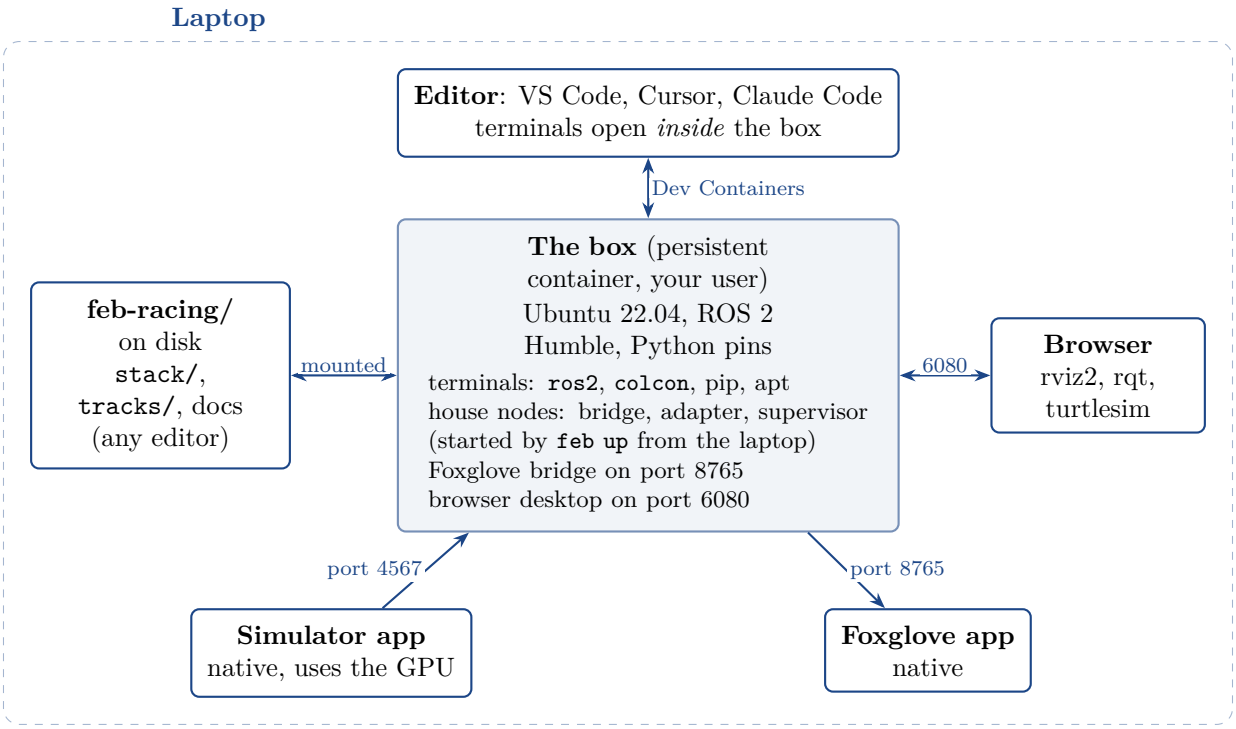
How the development environment should work, on the laptop and on the car

FEB autonomous team, 15 October 2026; built 16 October (`docs/devkit.md` is the how-to)

The idea in four lines

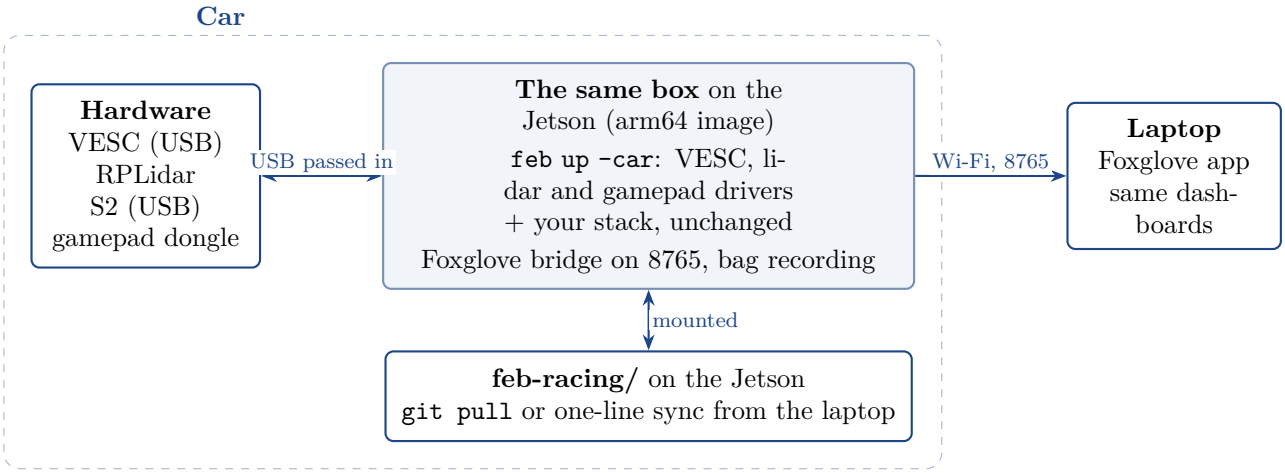
- Every member gets a **persistent Linux box**: Ubuntu 22.04 with ROS 2 Humble, their own user, state that survives. It is a Docker container, but nobody types `docker`.
- **Everything robotic runs inside the box**: terminals, builds, bags, the bridge, the stack, the car's drivers.
- The **simulator and Foxglove are apps on the laptop**, opened like a game. They find the box on their own.
- The **same box runs on the Jetson**. Code that drives in the sim drives the car, untouched.

1 On the laptop



What changed from today: the box is kept between sessions instead of rebuilt and deleted on every start; the editor's terminals are inside it; the sim is opened separately and finds the box; `feb-sim run` still exists as the one-shot race command.

2 On the car



The only difference between laptop and car is what runs *under* the stack: the sim bridge on the laptop, the real drivers on the Jetson. The stack never knows which.

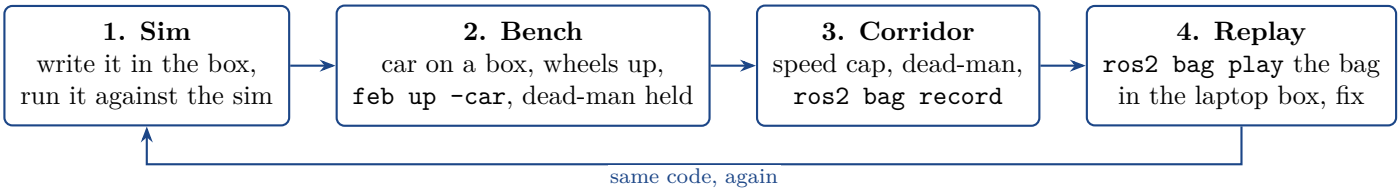
3 The contract that makes it one codebase

A stack reads two topics and writes one, the RoboRacer standard:

Topic	Type	Who publishes it
`/scan`	LaserScan	sim adapter (laptop) or RPLidar driver (car)
`/odom`	Odometry	sim adapter or VESC driver
`/drive`	AckermannDriveStamped	**your stack**; sim adapter or VESC driver consumes it

A small adapter node inside the box maps the simulator's `/autodrive/...` topics onto these. The current starter kit is moved onto the contract as the first user of it.

4 The testing ladder



5 A member's day, four ways

Just a Linux box to mess around in (headless)
Open `feb-racing` in the editor, click *Reopen in Container*. Every terminal is Ubuntu with ROS. Install things, write nodes, play bags. Close the editor, come back tomorrow, it is as you left it.

With a desktop (the onboarding lab)
Same box, gui flavour. One command puts a desktop in a browser tab for turtlesim, rqt_graph and rviz2. Laggy but fine for teaching.

Racing in the sim
From a laptop terminal, `./feb up -track loop -driver feb_driver`: house nodes, simulator and Foxglove in one go. Or start the house nodes by hand in a box terminal (`ros2 launch feb_tools up.launch.py`) and double-click the simulator.

On the car

Put the code on the Jetson (`git pull`). `./feb up -car -driver my_driver -mode-name inspection`. Hold the dead-man. Foxglove on the laptop at the Jetson's address.

6 Today and proposed, side by side

	Today	Proposed
Container	created and deleted on every `feb-sim run`	one persistent box per clone, `feb-sim reset` wipes it
User inside	root; files come out root-owned on Linux	your user
Builds	full `colcon build` on every start, 120 s deadline	incremental, no deadline
Installed packages	lost on stop	kept
Terminals	`./feb-sim shell` from a laptop terminal (the old launcher)	`./feb code`: editor terminals, already inside; `./feb shell` too
Starting the sim	launcher starts it	app you open, or the launcher, either order
Visualiser	browser desktop with rviz2 (optional Foxglove)	Foxglove app by default, browser desktop for the lab
Topics	simulator names, `/autodrive/roboracer_1/...`	`/scan`, `/odom`, `/drive` via an adapter
The car	follow the RoboRacer guide natively, port code by hand	same box, `feb up -car`
Submissions, league, CI	Docker image from the devkit	unchanged

7 What changed in the repo

- `feb` (was `feb-sim`): the box is a persistent named container; `up`, `shell`, `code`, `reset`, `logs`; `-ws`, `-driver`, `-mode-name`, `-car`.
- `devkit/Dockerfile`: user `feb`, sudo, node and the Claude CLI, the VESC driver, rplidar, joy, ackermann_msgs, scipy and casadi. `box.sh` is the box's process; `feb-up` builds a workspace and launches the house layer.
- `devkit/feb_tools`: `sim_adapter.py` (the contract from the simulator), `up.launch.py` (the house layer, sim or car), `car.launch.py` and `config/car.yaml` (the RoboRacer drivers).
- `devkit/feb_supervisor`: the state machine.
- `starter_kit`: `feb_driver` and `feb_cone_driver` on the contract; every package carries a `driver.yaml`.
- `docs/devkit.md` rewritten; the lab and the three competitor manuals updated.

8 Decisions

Files live on the laptop and are mounted in; VS Code is the standard editor and Cursor works too; Foxglove is the daily visualiser, the browser desktop stays for the lab and the odd rviz session; the box persists; `feb up` is the one-shot command; Docker Desktop is the engine, OrbStack allowed on Mac. Settled while building: all of our commands live on the laptop side as `feb`, inside the box there is only ROS and one launch file; the Jetson keeps its own clone of the FEB folder and is updated by `git pull`; the gui image is the box's image for everyone (one download), the base image stays the league's. Drivers carry a `driver.yaml` with modes (`inspection`, `trackdrive`, and whatever else the code does), and the supervisor is the only on-car manager: it decides whose command reaches the wheels.

Not yet done: a hardware test of `feb up -car` (the Jetson is not flashed), and the `feb_cone_racer` package still reads the simulator's topics directly.

9 Why not a VM

For the member, a VM and the box are the same thing: a Linux machine that keeps its state. The difference is the plumbing.

	The box	A VM
Starts in	a second	30 to 60 s
RAM	what is running	4 to 8 GB reserved
Disk	about 5 GB, shared image	20 to 40 GB per person
Install	Docker Desktop, then *Reopen in Container*	hypervisor, ISO, Ubuntu, ROS by hand; different per laptop type
Updating the base	`docker pull`, seconds	gigabytes again, or by hand
Apple Silicon	same image, arm64 built by CI	separate arm64 build; nobody ships one
Reaching the sim	done for you	port forwarding per member
Same as the car	yes	no, two environments
Submissions	it is the submission	still need Docker
Desktop	browser tab, laggy	native, nicer

The VM wins one row, the desktop, which Foxglove covers. Penn's course can recommend a VM because its simulator is a Python package that lives inside the VM; ours is a native GPU app that cannot. A VM recipe can still be published as an unsupported option for anyone who wants a full Ubuntu desktop; it changes nothing above.